Python Network Programming: Core Concepts Cheat Sheet

1. Socket Types & Basic Operations

Remember to `import socket` for all network code.

Operation	TCP (`socket.SOCK_STREAM`)	UDP (`socket.SOC`
Create Socket	`sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`	`sock = socket.so`

Server Setup	`sock.bind(...)`
	`sock.listen(...)`
	`conn, addr = sock.accept()`
		`sock.bind(...)`
		(No `listen` or `ac`

Client Setup	`sock.connect((host, port))`	*(No `connect` step*
Sending Data	`sock.sendall(data_bytes)`	`sock.sendto(data`
Receiving Data	`data = sock.recv(buffer_size)`	`data, sender_add`

---

2. Data Serialization & Integrity

All data sent over sockets must be `bytes`.

String-Based Formatting

Simple and human-readable. Good for text-based protocols.

```
# Create from variables
param1 = "value1"
param2 = 123
message_str = f"{param1}|{param2}|other_data"
message_bytes = message_str.encode()

# Parse from received bytes
data_str = received_bytes.decode()
parts = data_str.split('|') # -> ['value1', '123', 'other_data']
value1 = parts[0]
value2 = int(parts[1])
```

Binary Formatting (`struct`)

Efficient and fixed-size. Good for performance-critical or binary protocols. Remember `import struct`.

```
# Define a format: 'i' = integer, 'd' = double, '5s' = 5-char string
packer = struct.Struct('i d 5s')

# Pack data for sending
data_tuple = (123, 45.67, b'hello')
packed_bytes = packer.pack(*data_tuple)

# Unpack received data
unpacked_tuple = packer.unpack(received_bytes)
num = unpacked_tuple[0] # -> 123
```

Checksums & Hashes

For verifying data integrity.

- **CRC32 (Error Detection):** `import zlib`
  - `checksum = zlib.crc32(data_bytes)` — Returns an integer.
- **Cryptographic Hashes (Security):** `import hashlib`
  - `h = hashlib.sha256()` or `hashlib.md5()`
  - `h.update(data_bytes)`
  - `digest_str = h.hexdigest()` — Returns a hex string.

---

3. Handling Multiple Connections (`select`)

The standard pattern for concurrent, I/O-bound TCP servers. Remember `import select`.

```
# The list of sockets to monitor for incoming data
sockets_to_watch = [server_socket]

while True:
    # select() blocks until at least one socket is ready
    readable_sockets, _, _ = select.select(sockets_to_watch, [], [])

    for sock in readable_sockets:
        # If it's the main server socket, a new client is connecting
        if sock is server_socket:
            new_client_socket, client_address = server_socket.accept()
            sockets_to_watch.append(new_client_socket)

        # Otherwise, it's an existing client sending data
        else:
            data = sock.recv(1024)
            if data:
                # Process data from the client...
            else:
                # Client disconnected: stop watching and close the socket
                sockets_to_watch.remove(sock)
                sock.close()
```

---

4. Input/Output & Data Sources

Ways to get data into your program.

- **Command-Line Arguments:** `import sys`
  - `script_name = sys.argv[0]`
  - `first_arg = sys.argv[1]`
  - Remember to convert to `int()` or `float()` if needed.
- **Standard Input:**
  - `user_input = input("Prompt: ")`

- **Reading from a File:**

```
with open('config.txt', 'r') as f:
    content = f.read()
```

- **JSON Files:** `import json`

```
# Reading from a JSON file
with open('data.json', 'r') as f:
    config_dict = json.load(f)

# Writing to a JSON file (append logic)
# 1. Read the whole file into a list/dict.
# 2. Append/modify the list/dict in memory.
# 3. Write the entire updated structure back to the file.
with open('log.json', 'w') as f:
    json.dump(updated_data_list, f, indent=4)
```

---

5. Miscellaneous Tools

- **Random Data Generation:** `import random`
  - `random.randint(a, b)`: Get a random integer (inclusive).
  - `random.choice(my_list)`: Pick a random element from a list.
  - `random.sample(population, k)`: Pick `k` unique elements.